



NELUMBO
CONSULTORES

Reto Técnico de Ingreso Laboral

RETO NELUMBO

BACKEND

Estimado,

A continuación, describimos la especificación del reto que hemos diseñado para validar el proceso de ingreso a nuestra empresa. Las condiciones mínimas de aceptación son las indicadas en este documento, cualquier esfuerzo por demostrar capacidades superiores a las requeridas es bien valorado por nuestra empresa.

El reto a nivel técnico consiste en el desarrollo de la API con las funcionalidades y estructura indicadas a continuación.

¡Mucho éxito!

Nelumbo Consultores



Sistema de Gestión de Reservas para Salones de Eventos

1. CONSIDERACIONES GENERALES

- Se requiere una API REST para la gestión de reservas y control de ocupación de salones de eventos pertenecientes a múltiples administradores de sucursales.
- Se debe mantener un histórico de reservas finalizadas para la generación de indicadores y métricas operativas.
- La API debe contar con un sistema de autenticación protegido por JWT. El token tendrá una expiración de 6 horas.
- Se manejarán 2 roles: ADMIN y GESTOR.
- Control de aforo máximo por salón (no se permite reservar si excede capacidad ya comprometida en el mismo rango horario).
- Cálculo de cobro según horas reservadas y tarifa configurada por salón.
- Integración con un microservicio de notificaciones (simulado).

2. AUTENTICACIÓN Y SEGURIDAD

El sistema debe precargar un usuario administrador. Se manejará el email como identificador único de ingreso:

```
usuario: admin@mail.com  
pass: admin
```

El rol ADMIN es el único autorizado para crear usuarios con rol GESTOR.

Todos los endpoints deben estar protegidos por validación de token JWT y autorización por rol.

3. PERMISOLOGÍA

Rol ADMIN

- Crear usuarios con rol GESTOR.
- CRUD completo de sucursales y de salones (asociando un gestor responsable a cada sucursal).
- Consultar listado/detalle de reservas activas en cualquier salón.
- Enviar correos a los gestores (simulación vía microservicio).
- Visualizar todos los indicadores globales.
- Aprobar o rechazar reservas que requieran validación manual (ver sección 7).

Rol GESTOR

- Registrar reservas de salones pertenecientes a las sucursales que tenga asociadas.
- Registrar finalización (check-out) de una reserva, siempre y cuando exista una reserva activa para ese cliente y salón.

- Consultar listado de las sucursales y salones que tiene asociados.
- Consultar listado/detalle de las reservas activas en sus salones.
- Visualizar los indicadores asociados a sus salones.

4. FUNCIONALIDADES

4.1 Autenticación de Usuarios

Crear los endpoints necesarios para la autenticación de usuarios:

- POST /auth/login — Inicio de sesión, devuelve token JWT.
- POST /auth/register — Registro de usuarios (solo accesible por ADMIN para crear GESTORES).
- POST /auth/logout — Cierre de sesión / invalidación de token.

4.2 CRUD de Sucursales y Salones

Endpoints CRUD para sucursales y salones. Cada salón debe contener al menos:

- Nombre
- Sucursal a la que pertenece
- Capacidad máxima de personas
- Costo por hora
- Gestor asociado

4.3 Endpoint POST "registrar reserva"

A través del documento del cliente, id del salón, fecha-hora de inicio, fecha-hora de finalización estimada y número de asistentes. Internamente se registra la fecha-hora de creación de la reserva.

Respuesta exitosa (HTTP 201):

```
{
  "id": 0 // id generado del registro
}
```

Se debe validar que el documento ingresado no coincida con algún cliente que ya tenga una reserva activa en algún salón del sistema.

En caso contrario, retornar HTTP 400:

```
{
  "mensaje": "No se puede Registrar Reserva, ya existe una reserva activa para
este documento en este u otro salón"
}
```

Adicionalmente debe validarse que el número de asistentes no supere la capacidad disponible del salón en el rango horario solicitado. En tal caso retornar HTTP 400:

```
{
  "mensaje": "No se puede Registrar Reserva, capacidad insuficiente en el
salón"
}
```

4.4 Endpoint POST "finalizar reserva"

A través del documento del cliente y el id del salón. Internamente se registra la fecha-hora real de finalización y se calcula el cobro total según horas efectivas y tarifa del salón.

Respuesta exitosa (HTTP 200):

```
{
  "mensaje": "Reserva finalizada",
  "totalCobrado": 0
}
```

Se debe validar que la reserva esté activa al momento de la solicitud. Si no, retornar HTTP 400:

```
{
  "mensaje": "No se puede Finalizar Reserva, no existe una reserva activa para
este documento en el salón"
}
```

4.5 Endpoint GET "Listado de reservas activas"

Devuelve el listado de reservas que se encuentran actualmente activas en un salón específico.

Respuesta:

```
[
  {
    "id": 0,
    "documentoCliente": "",
    "nombreCliente": "",
    "fechaInicio": "",
    "fechaFinEstimada": "",
    "asistentes": 0,
    ...
  },
  {...}
]
```

4.6 Endpoint GET "Búsqueda por documento"

Buscar reservas activas por coincidencia parcial en el documento del cliente.

Ejemplo: al ingresar "105" debe retornar clientes con documento "1051234", "9101053", etc.

5. MICROSERVICIO DE NOTIFICACIONES (SIMULADO)

Se debe construir un microservicio independiente para simular el envío de notificaciones por correo.

Debe exponer un endpoint POST que reciba el siguiente body:

```
{
  "email": "",
  "documento": "",
  "mensaje": "",
  "salonNombre": ""
}
```

El microservicio debe:

- Imprimir en log la solicitud recibida.
- Retornar HTTP 200 con la siguiente respuesta:

```
{
  "mensaje": "Notificación Enviada"
}
```

Desde la API principal de reservas, se debe consumir este microservicio enviando los siguientes parámetros:

```
{
  "email": "",
  "documento": "",
  "mensaje": "",
  "salonId": ""
}
```

La API principal debe encargarse de transformar el salonId a salonNombre antes de llamar al microservicio, y debe retornar al cliente la respuesta que llega del microservicio.

Antes de invocar el microservicio, debe validarse que el documento del cliente tenga una reserva activa en el salón indicado.

Esquema de Integración



6. INDICADORES

- **(ADMIN y GESTOR)** Retornar los 10 clientes que más reservas han realizado en los diferentes salones del sistema, con el número total de reservas por cada uno.
- **(ADMIN y GESTOR)** Retornar los 10 clientes que más reservas han realizado en un salón específico, con su cantidad de reservas.
- **(ADMIN y GESTOR)** Verificar de las reservas activas, cuáles corresponden a clientes que reservan por primera vez en ese salón.

- **(GESTOR)** Obtener las ganancias generadas hoy, esta semana, este mes y este año en un salón específico.
- **(ADMIN)** Top 3 sucursales con mayor facturación en el mes actual.
- **(ADMIN y GESTOR)** Buscar reservas activas mediante coincidencia parcial en el documento del cliente. Ejemplo: ingreso "105" puede retornar clientes con documento "1051234" o "9101053".

7. REGLA DE NEGOCIO ADICIONAL — APROBACIÓN DE RESERVAS PREMIUM

Si una reserva supera un costo estimado de 500.000 (calculado como horas × costo por hora del salón), debe registrarse inicialmente en estado PENDIENTE_APROBACION y requerir aprobación manual del rol ADMIN antes de considerarse activa.

Reglas del flujo de aprobación:

- El estado inicial será PENDIENTE_APROBACION.
- Las reservas en este estado NO ocupan cupo del salón hasta ser aprobadas.
- Existirá un endpoint POST /reservas/{id}/aprobar (exclusivo ADMIN) que transiciona el estado a ACTIVA y dispara una notificación al gestor responsable usando el microservicio simulado.
- Existirá un endpoint POST /reservas/{id}/rechazar (exclusivo ADMIN) que transiciona el estado a RECHAZADA con un motivo obligatorio en el body.
- Las reservas con más de 48 horas en PENDIENTE_APROBACION deben marcarse automáticamente como EXPIRADA (puede implementarse mediante validación al consultar o mediante tarea programada).
- Una reserva RECHAZADA o EXPIRADA no puede reactivarse; el cliente deberá generar una nueva reserva.

8. CONSIDERACIONES A TENER EN CUENTA

- Cada salón se configura en su creación con su capacidad máxima de personas y su costo por hora.
- Validar siempre la capacidad disponible al momento del registro de la reserva (error 400 con mensaje descriptivo si excede).
- El documento del cliente debe tener entre 6 y 12 caracteres, únicamente numéricos. No se permiten caracteres especiales, letras ni espacios.
- El número de asistentes debe ser un entero positivo y nunca cero.
- La fecha de fin estimada debe ser estrictamente posterior a la fecha de inicio.
- Cuando se finaliza una reserva, se debe mover a una tabla de histórico junto con su fecha real de finalización y el cobro total calculado.

- El cálculo de horas debe redondear hacia arriba: cualquier fracción menor a una hora se cobra como una hora completa.

9. OBSERVACIONES Y REQUISITOS TÉCNICOS

- Usar excepciones personalizadas cuando se requiera retornar respuestas con error 400 detectadas dentro de las clases de servicio.
- Implementar arquitectura en capas con separación clara: controllers, services, entities, repositories, donde cada capa cumpla su responsabilidad.
- Manejar la protección de los endpoints por rol y validación de token JWT.
- Usar PostgreSQL como motor de base de datos.
- Usar Java SpringBoot.
- Aplicar el patrón DTO para las entradas y salidas de los controllers, separándolos de las entidades de persistencia.
- Entregar colección de Postman con cada request documentado (incluyendo headers, body y ejemplos de respuesta).
- Definir la estructura de paquetes del código de forma clara y consistente.
- Plantear y entregar un modelo entidad-relación de la solución (puede ser imagen o diagrama embebido en el README).
- Incluir manejo global de excepciones (@ControllerAdvice) para respuestas de error consistentes.
- Para la entrega, proporcionar un enlace al repositorio en GitHub con la solución, y en el README incluir cualquier observación necesaria para ejecutar el proyecto en local (variables de entorno, scripts SQL iniciales, comandos de arranque, etc.).

10. ASPECTOS QUE SUMAN VALOR ADICIONAL (OPCIONALES)

Los siguientes elementos no son obligatorios pero serán bien valorados por nuestra empresa:

- Pruebas unitarias y/o de integración con cobertura razonable (JUnit, Mockito).
- Dockerización del proyecto (Dockerfile + docker-compose para PostgreSQL y servicios).
- Documentación de la API con Swagger/OpenAPI.
- Implementación de logs estructurados y manejo de niveles (INFO, WARN, ERROR).
- Migraciones de base de datos con Flyway o Liquibase.
- Validaciones con Bean Validation (@Valid, @NotNull, @Size, etc.).